100 DevOps Interview Questions & Answers

A complete reference covering Linux, Git, CI/CD, Docker, Kubernetes, Terraform, AWS/Azure, GitOps, Observability, Security, and Behavioral rounds.

Compiled by Rushi | dpstudios.in

1. Linux & Fundamentals

Q1. What is the difference between a hard link and a soft link?

A hard link points directly to the same inode as the original file — deleting the original doesn't break it, and both share the same data on disk. A soft (symbolic) link is a pointer to the file path; if the original is deleted or moved, the symlink breaks. Hard links can't cross filesystems or link to directories; symlinks can.

Q2. How do you check which process is using a specific port?

Use `sudo lsof -i :PORT` or `sudo netstat -tulpn | grep PORT` or the modern equivalent `sudo ss -tulpn | grep PORT`. This shows the PID and process name bound to that port, useful for resolving 'address already in use' errors.

Q3. Explain the Linux boot process.

BIOS/UEFI POST → bootloader (GRUB) loads the kernel → kernel initializes hardware and mounts root filesystem → kernel starts init/systemd (PID 1) → systemd starts targets/services in dependency order → login prompt or display manager loads.

Q4. What's the difference between a process and a thread?

A process is an independent execution unit with its own memory space; threads are lightweight units within a process that share the same memory space but have their own stack. Threads are cheaper to create and communicate faster, but a crash in one can affect the whole process.

Q5. How do you find and kill a process consuming high CPU?

Use `top` or `htop` to identify the PID by CPU%, then `kill -15 PID` for a graceful shutdown or `kill -9 PID` to force kill. `ps aux --sort=-%cpu | head` is a quick one-liner to list top consumers.

Q6. What are file permissions 755 and 644 used for?

755 (rwxr-xr-x) is typical for executables/scripts and directories — owner has full access, group and others can read/execute. 644 (rw-r--r--) is typical for regular files — owner can read/write, others can only read.

Q7. Explain the difference between TCP and UDP.

TCP is connection-oriented, guarantees ordered and reliable delivery via handshakes and acknowledgments — used for HTTP, SSH, databases. UDP is connectionless, faster, with no delivery guarantee — used for DNS, video streaming, and cases where speed matters more than reliability.

Q8. What is a zombie process and how do you handle it?

A zombie is a process that has completed execution but still has an entry in the process table because its parent hasn't read its exit status. It doesn't consume resources itself; you fix it by getting the parent process to call wait(), or by killing the parent if it's misbehaving.

2. Git & Version Control

Q9. What's the difference between git merge and git rebase?

Merge creates a new commit combining two branch histories, preserving full history but adding merge commits. Rebase replays your commits on top of the target branch, producing a linear history — cleaner but rewrites commit hashes, so avoid rebasing shared/public branches.

Q10. How do you resolve a merge conflict?

Git marks conflicting sections with <<<<<<<, =======, >>>>>>> markers in the file. You manually edit to keep the correct code, remove the markers, then `git add` the resolved file and `git commit` (or `git rebase --continue` if mid-rebase).

Q11. What is git cherry-pick and when would you use it?

It applies a specific commit from one branch onto another without merging the whole branch. Useful for hotfixes — picking a single bug-fix commit into a release branch without pulling in unrelated changes.

Q12. Explain git reset vs git revert.

`git reset` moves the branch pointer backward, optionally discarding commits (--hard) or keeping changes staged/unstaged (--soft/--mixed) — rewrites history, unsafe on shared branches. `git revert` creates a new commit that undoes a previous commit's changes — safe for shared history.

Q13. What is a detached HEAD state?

It happens when you check out a specific commit instead of a branch — HEAD points directly to a commit, not a branch reference. Any new commits made here aren't attached to a branch and can be lost unless you create a new branch to save them.

Q14. How do you undo the last commit but keep the changes?

`git reset --soft HEAD~1` removes the commit but keeps changes staged. Use `--mixed` (default) to unstage them too, or `--hard` only if you want to discard the changes entirely.

3. CI/CD Pipelines

Q15. What is the difference between Continuous Delivery and Continuous Deployment?

Continuous Delivery means every change that passes automated tests is deployable to production but requires a manual approval/trigger to actually release. Continuous Deployment goes further — every passing change is automatically deployed to production with no human gate.

Q16. Walk me through a CI/CD pipeline you built end to end.

A strong answer covers: source trigger (push/PR), build stage, unit/static analysis tests, artifact creation and versioning, container image build and push to registry, deployment to a staging environment, integration tests, manual or automated gate, then production deployment — plus rollback strategy.

Q17. How do you handle secrets in a CI/CD pipeline?

Never hardcode secrets in pipeline YAML or source. Use a secrets manager (Azure Key Vault, AWS Secrets Manager, HashiCorp Vault) or the CI tool's native secret store (GitHub Actions secrets, Jenkins credentials), inject at runtime as environment variables, and restrict access via least-privilege service accounts.

Q18. What's the difference between GitHub Actions and Jenkins?

Jenkins is a self-hosted, plugin-driven automation server offering full control but requiring you to manage infrastructure and updates. GitHub Actions is a managed, YAML-based CI/CD tightly integrated with GitHub repos with less operational overhead but less flexibility than a fully customized Jenkins setup.

Q19. How do you implement a rollback strategy in your pipeline?

Common approaches: keep the last N versioned artifacts/images and redeploy the previous known-good one, use blue-green or canary deployments so you can switch traffic back instantly, or trigger an automated rollback on failed health checks post-deployment.

Q20. What are pipeline stages and how do you structure them for speed?

Typical stages: build → test → security scan → package → deploy. To speed things up: run independent jobs in parallel, cache dependencies between runs, use matrix builds for multi-environment testing, and fail fast on cheap checks (lint/unit tests) before expensive ones (integration tests).

Q21. How do you manage environment-specific configuration across dev/staging/prod?

Externalize config from code using environment variables, config maps, or a parameter store (AWS SSM, Azure App Configuration). Keep environment-specific values out of the repo, use templating (Helm values files, Terraform workspaces) to inject the right values per environment.

Q22. What is a canary deployment and how do you implement one?

A canary deployment routes a small percentage of live traffic to the new version while most traffic still hits the stable version, monitoring error rates/latency before gradually increasing traffic. Implemented via service mesh traffic splitting (Istio), weighted load balancer rules, or Kubernetes with tools like Argo Rollouts.

Q23. Explain blue-green deployment.

Two identical production environments (blue = current, green = new) run in parallel. Traffic is switched from blue to green all at once (via load balancer or DNS) after green is verified healthy, giving near-instant rollback by switching back if issues arise.

Q24. How do you handle database migrations in a CI/CD pipeline?

Use versioned migration tools (Flyway, Liquibase, EF Migrations) run as a separate pipeline step before app deployment. Migrations should be backward-compatible so the old app version keeps working during rollout, and destructive changes (drop column) are done in a later, separate release.

4. Docker & Containers

Q25. What is the difference between an image and a container?

An image is a read-only, immutable template containing application code, dependencies, and filesystem layers. A container is a running (or stopped) instance of that image with its own writable layer, network, and process space.

Q26. How do you reduce Docker image size?

Use minimal base images (alpine, distroless), multi-stage builds to discard build tools from the final image, combine RUN commands to reduce layers, clean up package manager caches in the same layer, and use .dockerignore to exclude unnecessary files.

Q27. Explain multi-stage builds and why they matter.

Multi-stage builds use multiple FROM statements in one Dockerfile — one stage compiles/builds the app with full tooling, and a final stage copies only the compiled artifact into a minimal runtime image. This drastically shrinks final image size and attack surface.

Q28. What's the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the fixed command that always runs when the container starts; CMD provides default arguments that can be overridden at runtime. Combining both (ENTRYPOINT for the binary, CMD for default flags) is a common pattern.

Q29. How do containers achieve isolation without a full hypervisor?

Containers use Linux kernel features — namespaces (PID, network, mount, UTS, IPC) for isolation of what a process can see, and cgroups to limit and account for resource usage (CPU, memory, I/O). They share the host kernel, unlike VMs which virtualize hardware.

Q30. How do you persist data in Docker containers?

Use volumes (managed by Docker, stored outside the container's writable layer) for production data, or bind mounts to map a host directory into the container for development. Volumes survive container removal; the container's own writable layer doesn't.

Q31. What happens when a container's main process exits?

The container stops, since Docker containers are tied to the lifecycle of PID 1 inside them. If PID 1 exits (crashes or completes), the whole container exits, even if there are other background processes still running inside it.

Q32. How do you debug a container that keeps crashing (CrashLoopBackOff-style behavior)?

Check `docker logs <container>` for the exit error, run the image interactively with `docker run -it --entrypoint /bin/sh <image>` to inspect the filesystem, verify environment variables and mounted config, and check resource limits — OOM kills often look like crashes.

5. Kubernetes

Q33. What is the difference between a Deployment and a StatefulSet?

A Deployment manages stateless, interchangeable pod replicas with no stable identity or storage guarantees. A StatefulSet gives each pod a stable network identity and persistent storage that survives rescheduling — used for databases, message queues, and other stateful workloads.

Q34. Explain the Kubernetes control plane components.

kube-apiserver (front door for all API requests), etcd (cluster state store), kube-scheduler (assigns pods to nodes), kube-controller-manager (runs reconciliation loops like ReplicaSet and Node controllers), and cloud-controller-manager (integrates with cloud provider APIs).

Q35. What is a readiness probe vs a liveness probe?

A liveness probe checks if a container is still alive; if it fails, Kubernetes restarts the container. A readiness probe checks if a container is ready to receive traffic; if it fails, the pod is removed from service endpoints but not restarted — useful during startup or temporary overload.

Q36. How does a Kubernetes Service route traffic to pods?

A Service selects pods via label selectors and gets a stable virtual IP (ClusterIP). kube-proxy programs iptables or IPVS rules on each node so traffic to the Service IP gets load-balanced to healthy pod endpoints, which are continuously tracked via the Endpoints/EndpointSlice objects.

Q37. What's the difference between a ConfigMap and a Secret?

Both store configuration data injectable into pods as env vars or volumes. ConfigMaps hold non-sensitive plain-text config; Secrets hold sensitive data and are base64-encoded (not encrypted by default at rest unless you enable encryption at the etcd level or use an external secrets manager).

Q38. Explain how Horizontal Pod Autoscaler (HPA) works.

HPA watches metrics (CPU, memory, or custom metrics via metrics-server/Prometheus adapter) against a target threshold on a Deployment/StatefulSet, and automatically increases or decreases the replica count to maintain that target, within configured min/max bounds.

Q39. What is the difference between a DaemonSet and a Deployment?

A DaemonSet ensures exactly one pod runs on every (or selected) node — used for node-level agents like log collectors or monitoring exporters. A Deployment manages a desired number of replicas scheduled wherever the scheduler decides, independent of node count.

Q40. How do you troubleshoot a pod stuck in Pending state?

`kubectl describe pod` shows scheduling events — common causes are insufficient CPU/memory on nodes, unsatisfied node affinity/taints, unbound PersistentVolumeClaims, or no nodes matching the pod's selector. Check `kubectl get events` and node resource availability too.

Q41. What is a taint and toleration?

A taint is applied to a node to repel pods that don't explicitly tolerate it (e.g., dedicating nodes to specific workloads). A toleration is applied to a pod, allowing (but not forcing) it to be scheduled on nodes with a matching taint.

Q42. Explain Kubernetes namespaces and when to use them.

Namespaces partition a cluster into virtual sub-clusters for resource organization, access control (via RBAC), and resource quotas — typically used to separate teams, environments (dev/staging/prod), or applications within a shared cluster.

Q43. What is an Ingress and how is it different from a Service?

A Service (type LoadBalancer/NodePort) exposes a single application's network access, typically at L4. An Ingress is an L7 routing rule set (host/path-based routing, TLS termination) managed by an Ingress Controller (nginx, ALB), letting multiple services share a single external entry point.

Q44. How would you troubleshoot high memory usage causing pod OOMKills?

Check `kubectl describe pod` for OOMKilled status and exit code 137, review resource requests/limits versus actual usage via metrics-server or Prometheus, check for memory leaks in application logs, and consider raising limits or optimizing the app's memory footprint.

Q45. What is the difference between resource requests and limits?

Requests are what the scheduler guarantees and reserves for a container — used for bin-packing decisions. Limits are the hard ceiling a container cannot exceed; exceeding a memory limit triggers an OOMKill, while exceeding a CPU limit causes throttling, not a kill.

Q46. Explain how you'd perform a zero-downtime rolling update in Kubernetes.

Set an appropriate `maxUnavailable` and `maxSurge` in the Deployment's RollingUpdate strategy, ensure readiness probes are properly configured so traffic isn't sent to unready pods, and use `preStop` hooks with graceful termination periods so in-flight requests complete before a pod is killed.

Q47. What is a PodDisruptionBudget and why use one?

A PDB limits how many pods of an application can be voluntarily disrupted at once (during node drains, cluster upgrades, or autoscaling events), ensuring a minimum number stay available to prevent an outage during maintenance operations.

6. Terraform & Infrastructure as Code

Q48. What is Terraform state and why is it important?

State is a JSON file mapping your configuration to real-world resources, tracking metadata and dependencies. Terraform uses it to know what exists, detect drift, and plan changes accurately — without it, Terraform can't determine what to create, update, or destroy.

Q49. How do you manage Terraform state in a team environment?

Use a remote backend (S3 + DynamoDB for locking, Azure Storage with state locking, or Terraform Cloud) so state is shared and locked to prevent concurrent writes, rather than relying on local state files that cause conflicts and drift.

Q50. What is the difference between terraform plan and terraform apply?

`plan` shows a preview of what changes will be made (create/update/destroy) without touching infrastructure — used for review. `apply` executes those changes against real infrastructure, and by default re-runs a plan first for confirmation.

Q51. Explain Terraform modules and why you'd use them.

Modules are reusable, self-contained groups of resources with defined inputs/outputs, similar to functions in programming. They let you standardize infrastructure patterns (e.g., a 'VPC module' or 'AKS cluster module') and reuse them across projects and environments.

Q52. What is state drift and how do you handle it?

Drift happens when real infrastructure changes outside of Terraform (manual console edits, other tools). Run `terraform plan` to detect drift, then either `terraform apply` to bring infra back in line with code, or `terraform import`/update code to reflect the manual change as the new source of truth.

Q53. How do you handle secrets in Terraform code?

Never hardcode secrets in .tf files or commit them to version control. Use variables marked `sensitive = true`, pull values from a secrets manager (Vault, AWS Secrets Manager) via data sources at runtime, and ensure state files (which may contain sensitive values) are stored encrypted with restricted access.

Q54. What's the difference between Terraform and Ansible?

Terraform is declarative infrastructure provisioning — it creates and manages the lifecycle of cloud resources. Ansible is primarily configuration management/orchestration — installing software, changing config on existing servers. Many teams use Terraform to provision infra and Ansible to configure it.

Q55. Explain the purpose of terraform.tfstate locking.

Locking prevents two people or pipeline runs from applying changes simultaneously, which could corrupt state or cause conflicting resource modifications. Remote backends like S3+DynamoDB or Terraform Cloud handle this automatically during plan/apply operations.

Q56. What is a Terraform workspace and when would you use one?

Workspaces let you maintain multiple distinct state files from the same configuration — useful for managing dev/staging/prod with the same code but separate state. For larger differences between environments, separate directories/modules are usually cleaner than workspaces alone.

Q57. How do you safely destroy or recreate a specific resource without affecting others?

Use `terraform destroy -target=resource.name` for destruction or `terraform taint resource.name` (or `-replace` in newer versions) to force recreation on the next apply, scoping the blast radius to just that resource instead of the whole state.

7. Cloud (AWS & Azure)

Q58. What is the difference between an AWS Security Group and a NACL?

A Security Group is a stateful, instance-level firewall — return traffic is automatically allowed. A Network ACL is a stateless, subnet-level firewall where you must explicitly allow both inbound and outbound rules, evaluated in rule-number order.

Q59. Explain how Auto Scaling Groups work in AWS.

An ASG maintains a desired number of EC2 instances based on scaling policies tied to CloudWatch metrics (CPU, request count) or schedules. It automatically launches new instances from a launch template when scaling out, and terminates instances (respecting termination policies) when scaling in.

Q60. What is the difference between an Application Load Balancer and a Network Load Balancer?

ALB operates at L7, routing based on HTTP path/host headers, ideal for microservices and content-based routing. NLB operates at L4, handling raw TCP/UDP with ultra-low latency and static IPs, ideal for extreme performance or non-HTTP protocols.

Q61. How do you secure access to an EKS cluster?

Combine IAM authentication (aws-auth ConfigMap mapping IAM roles to Kubernetes RBAC), private API server endpoints or restricted CIDR access, Kubernetes RBAC for fine-grained permissions, and network policies to control pod-to-pod traffic.

Q62. What is the difference between Azure AKS and self-managed Kubernetes?

AKS is a managed Kubernetes service where Azure handles the control plane (masters, etcd, upgrades) at no extra charge for the control plane itself — you only manage worker nodes and workloads. Self-managed Kubernetes means you provision and maintain every component, including the control plane, yourself.

Q63. Explain the AWS shared responsibility model.

AWS is responsible for security 'of' the cloud — physical infrastructure, hardware, and the underlying virtualization layer. The customer is responsible for security 'in' the cloud — data, IAM configuration, OS patching (for EC2), network configuration, and application-level security.

Q64. What is the difference between Azure Resource Manager (ARM) templates and Terraform?

ARM templates are Azure-native, JSON-based, and tightly integrated with Azure resource providers with no third-party dependency. Terraform is cloud-agnostic, uses HCL, and can manage multi-cloud and third-party resources in one workflow, at the cost of an extra provider abstraction layer.

Q65. How would you design a highly available, multi-AZ architecture on AWS?

Deploy across at least two/three Availability Zones with an ALB distributing traffic, an Auto Scaling Group spanning those AZs, a Multi-AZ RDS instance (or read replicas) for the database layer, and S3/CloudFront for static assets — eliminating single points of failure at each tier.

Q66. What is IAM least privilege and how do you enforce it practically?

Least privilege means granting only the exact permissions needed to perform a task, nothing more. In practice: use scoped IAM policies instead of wildcard actions/resources, prefer roles over long-lived access keys, regularly audit with IAM Access Analyzer, and use permission boundaries for delegated admin.

Q67. What is the difference between an S3 bucket policy and an IAM policy?

An IAM policy is attached to a user/role/group and defines what that identity can do across AWS. A bucket policy is attached directly to the S3 bucket and defines who (which principals) can access it — useful for cross-account access or public access control, evaluated together with IAM policies.

8. GitOps & ArgoCD

Q68. What is GitOps and how does it differ from traditional CI/CD push deployment?

GitOps treats Git as the single source of truth for desired infrastructure/application state; an in-cluster agent (like ArgoCD) continuously pulls and reconciles the cluster to match Git. Traditional CI/CD pushes changes out to the cluster from the pipeline — GitOps pulls changes in, giving auditability and easy rollback via Git history.

Q69. How does ArgoCD detect and handle configuration drift?

ArgoCD continuously compares the live cluster state against the desired state defined in Git. If they diverge (someone made a manual kubectl change), it marks the application 'OutOfSync' and — depending on sync policy — either alerts you or automatically re-syncs to bring the cluster back to match Git.

Q70. What is the difference between automated sync and manual sync in ArgoCD?

Manual sync requires a person or external trigger to approve applying Git changes to the cluster, giving a review gate. Automated sync applies changes as soon as they're detected in Git with no human step, which is faster but riskier without strong pre-merge review and testing.

Q71. How do you roll back a bad deployment in a GitOps model?

Revert the offending commit in Git (via `git revert` to preserve history) and let ArgoCD/Flux automatically re-sync the cluster to the previous known-good state — the rollback is a Git operation, not a manual kubectl/helm command against the cluster.

9. Monitoring, Logging & Observability

Q72. What's the difference between monitoring and observability?

Monitoring is watching known metrics/dashboards to detect predefined problems. Observability is the broader ability to ask arbitrary questions about system behavior from logs, metrics, and traces to debug unknown/unforeseen issues — monitoring tells you something is wrong, observability helps you find out why.

Q73. Explain the three pillars of observability.

Metrics (numeric time-series data like CPU%, request rate), Logs (discrete timestamped event records), and Traces (end-to-end request paths across distributed services). Together they let you detect, diagnose, and root-cause issues across complex systems.

Q74. How does Prometheus collect metrics, and what is its data model?

Prometheus uses a pull model — it scrapes HTTP /metrics endpoints on targets at configured intervals. Data is stored as time-series identified by a metric name plus key-value labels, enabling flexible querying and aggregation via PromQL.

Q75. What is the purpose of alerting rules and how do you avoid alert fatigue?

Alerting rules trigger notifications when metrics cross defined thresholds for a sustained period. To avoid fatigue: alert on symptoms that affect users (not every internal metric), use appropriate 'for' durations to avoid flapping, tune severity levels, and route non-urgent alerts to lower-priority channels.

Q76. How would you set up centralized logging for a Kubernetes cluster?

Deploy a log-shipping agent as a DaemonSet (Fluentd, Fluent Bit, or Promtail) on every node to collect container/pod logs, forward to a centralized store (Elasticsearch, Loki, or a cloud-native service), and visualize/query via Kibana or Grafana.

10. Security & DevSecOps

Q77. What is the principle of 'shift-left security'?

It means integrating security checks earlier in the development lifecycle — static analysis, dependency scanning, and secret detection run at commit/PR time rather than only pre-production — catching vulnerabilities cheaply before they reach production.

Q78. How do you scan container images for vulnerabilities in a pipeline?

Integrate a scanner (Trivy, Grype, or a registry-native scanner like ECR/ACR scanning) as a pipeline stage after the image build, fail the build on critical/high CVEs above a defined threshold, and re-scan periodically since new CVEs are discovered against existing images.

Q79. What is SAST vs DAST?

SAST (Static Application Security Testing) analyzes source code for vulnerabilities without executing it, run early in CI. DAST (Dynamic Application Security Testing) tests a running application from the outside, simulating real attacks — run later against a deployed staging environment.

Q80. How do you manage secrets rotation in production systems?

Use a secrets manager (Vault, AWS Secrets Manager, Azure Key Vault) that supports automated rotation, configure applications to fetch secrets dynamically rather than baking them into images/config, and ensure rotation doesn't cause downtime by supporting overlapping validity windows for old/new credentials.

11. Networking & Config Management

Q81. What is DNS and how does resolution work at a high level?

DNS translates human-readable domain names into IP addresses. A resolver queries a root server → TLD server (.com, .in) → authoritative nameserver for the domain, caching results along the way based on TTL to speed up future lookups.

Q82. Explain the difference between a forward proxy and a reverse proxy.

A forward proxy sits in front of clients, forwarding their requests to the internet and hiding client identity (used for filtering/anonymity). A reverse proxy sits in front of servers, receiving client requests and routing them to backend servers — used for load balancing, TLS termination, and caching (e.g., nginx, HAProxy).

Q83. What is Ansible and how does it differ from Terraform in your workflow?

Ansible is agentless configuration management that connects over SSH to configure existing servers — installing packages, managing files, and running tasks idempotently via YAML playbooks. In practice, Terraform provisions the infrastructure and Ansible configures the software on top of it.

Q84. What does idempotency mean in the context of infrastructure automation, and why does it matter?

An idempotent operation produces the same end state no matter how many times it's run. It matters because pipelines and playbooks often re-run after partial failures — idempotency ensures re-running doesn't create duplicate resources or break existing state.

Q85. How do you troubleshoot a service that's unreachable over the network?

Work through the layers: check if the service process is running (`systemctl status`), check if it's listening on the expected port (`ss -tulpn`), check local firewall rules, check security group/NACL rules if cloud-hosted, then test connectivity from the client side with `curl`/`telnet`/`nc`.

Q86. What is the difference between a load balancer's Layer 4 and Layer 7 operation?

Layer 4 load balancing routes based on IP address and port without inspecting the actual content of the traffic — fast, protocol-agnostic. Layer 7 load balancing inspects the application-layer content (HTTP headers, URL paths, cookies) to make smarter routing decisions, at the cost of more processing overhead.

Q87. Explain the concept of a subnet and CIDR notation.

A subnet is a logical subdivision of a network. CIDR notation (e.g., 10.0.0.0/24) specifies the network address plus a prefix length indicating how many bits are fixed for the network portion — /24 means 256 addresses, with the remaining 8 bits available for host addresses.

Q88. What is an Ansible playbook and how is idempotency achieved in it?

A playbook is a YAML file defining a series of tasks to run against target hosts. Idempotency comes from Ansible modules checking current state before acting — e.g., the `package` module only installs if the package is absent, so re-running the playbook doesn't reinstall or error out.

12. Scenario & Behavioral

Q89. A production deployment just broke the site. What are your first three steps?

First, roll back immediately to the last known-good version to restore service (don't debug in production while users are affected). Second, communicate status to stakeholders. Third, once stable, investigate root cause using logs/metrics from the failed deployment window.

Q90. How do you approach an on-call incident where you don't immediately know the cause?

Stabilize first (rollback, scale up, or failover) before diagnosing. Check recent changes (deployments, config changes) as the most likely cause, review dashboards for anomalies at the time of impact, and escalate early rather than debugging alone under pressure if the impact is severe.

Q91. Describe a time you improved a CI/CD pipeline's performance.

A strong answer names a concrete before/after (e.g., 'build went from 12 minutes to 4') and the specific technique used — dependency caching, parallelizing test suites, using a leaner base image, or splitting a monolithic pipeline into independent jobs.

Q92. How do you decide what should be automated versus done manually?

Automate anything repetitive, error-prone when done by hand, or time-sensitive (deployments, scaling, backups). Keep manual gates for high-risk, infrequent, or judgment-requiring actions (production database schema changes, major architecture shifts) until enough confidence is built to automate safely.

Q93. How do you keep infrastructure costs under control in a cloud environment?

Right-size resources based on actual utilization data, use auto-scaling instead of static over-provisioning, leverage spot/reserved instances for predictable or fault-tolerant workloads, set up cost alerts/budgets, and regularly audit for unused resources (orphaned volumes, idle load balancers).

Q94. What's your approach to writing documentation for infrastructure you build?

Document the 'why' not just the 'what' — architecture decisions and trade-offs, not just command lists. Keep it close to the code (README in the repo, comments in Terraform modules) so it stays in sync, and include a runbook for common failure scenarios.

Q95. How do you handle a disagreement with a developer about a deployment process?

Focus on the shared goal (reliable, fast delivery) rather than the disagreement itself, back your position with data (deployment failure rates, incident history) rather than opinion, and be open to their constraints — a good process has to work for both dev velocity and operational stability.

Q96. What metrics do you use to measure DevOps success?

The DORA metrics are the industry standard: deployment frequency, lead time for changes, change failure rate, and mean time to recovery (MTTR). Together they balance speed (how fast you ship) against stability (how safely you ship).

Q97. How do you plan and execute a zero-downtime infrastructure migration (e.g., moving a cluster or database)?

Run old and new environments in parallel, replicate data continuously to the new system, gradually shift traffic (via DNS weighting or load balancer rules) while monitoring error rates, and keep the old environment as a fallback until the new one is proven stable under full production load.

Q98. What was the most difficult production outage you've handled, and what did you learn?

A strong answer names the specific failure, the diagnostic steps taken under pressure, the fix, and — most importantly — a concrete process or tooling change made afterward (better alerting, a new runbook, added redundancy) so the same failure mode couldn't recur silently.

Q99. How do you stay current with the fast-moving DevOps/cloud landscape?

A credible answer names specific habits — following release notes for tools you use daily (Kubernetes, Terraform providers), hands-on labs for new certifications, and applying new patterns in side projects before adopting them in production — rather than a vague 'I read a lot.'

Q100. Where do you see the DevOps role evolving over the next few years?

Reasonable talking points: platform engineering absorbing much of traditional DevOps into self-service internal developer platforms, deeper AI-assisted automation in incident response and code review, and continued consolidation around GitOps and policy-as-code as the default operating model.